自作言語で作った自作 CPU の上で自作 Ruby が動かない

全部自分のコードのとき、どこを疑うか

v0: 時間を考えない全部入り。当日版は、ここからの引き算

284km ・ YAPC::Tokyo 2026

パッチワーク

YAPC::Tokyo 2026 のテーマ。「小さな布を縫い合わせて、1 つの大きな作品にする」

・差分を当てる道具 patch を書いたのは、Larry Wall（1985 年）

・今日の塔も、自分で書いた小さな布の縫い合わせ

継ぎ当てについて、Larry Wall が書いたこと

> No one sews a patch of unshrunk cloth on an old garment, for the patch will pull away from the garment, making the tear worse.

2019 年 10 月 11 日、Perl 6 を Raku に改名する提案への、賛成のコメント

今日の話は、継ぎ当てが引きはがれた話です

今日の話

・作ったものの紹介ではありません

・疑える層がいくつもあるとき、どこから疑うか

・推測を、検査に変えるやり方

・そして、検査そのものに騙された話

コンパイラ・CPU・言語処理系の知識は要りません

第０章　塔

全部、自分で書いた

・自作言語 Mere（OCaml で書いたコンパイラ）

・出力先は 5 つ: 解釈実行、C、LLVM、Wasm、RISC-V の機械語

・RISC-V の CPU エミュレータ（Mere で書いた。32 ビットと 64 ビット）

・その上で動く、自作の OS と、スケジューラと、シェル

・Ruby 処理系（Mere で書いた。4 万行超）

道具も、だいたい自分で書いた

- 圧縮（gzip 互換）、PNG、JPEG
- TrueType フォント、ラスタライズ、日本語の改行位置
- Markdown、スライド作成（このスライド）
- ブラウザ、3D 描画、テキストエディタ、コンテナ基盤

自作の層しかない、ということ

・他人の層なら、材料が外から来る: issue、既知のバグ、「みんな使っている」

・自分の層だと、それが 1 つも来ない

・下の層を疑わない、という定石が使えない

最初に、ひとつ選んでください

次の章に出てくる 3 つの容疑者のうち、犯人はどれか

章の終わりで答え合わせをします

第１章　puts 1 + 1

CPU を 32 ビットから 64 ビットに広げた

puts 1 + 1

が通らない

手を入れたのは、出す側と動かす側の両方

- 機械語を出すコンパイラと、それを動かす CPU エミュレータ
- 書き換えは 149 箇所
- 1 つのコミットではない

定石どおり、境界はテストしてあった

層	何と比べたか	結果
1. Ruby 処理系	本家 ruby 3.2.2、162 本	バイト単位で一致
2. 機械語	同じプログラムを C に変換して実行	64 / 32 ビットとも不一致 0
3. CPU エミュレータ	同じ機械語を QEMU で実行	7 / 7 一致

2026 年 9 月初め。全部、緑

境界のテストが言えること

試したプログラムでは、一致する

「その層が正しい」とは、言っていない

容疑者は 3 人とも残る

1. 自作 Ruby 処理系
2. 自作コンパイラが出した機械語
3. 自作 CPU エミュレータ

どれだと思いますか

見当 1: 直前に手を入れた場所

・64 ビット化で、関数ごとの「変数の置き場」の大きさを書き換えた

・壊れた値の置き場所を追うと、関数が確保した範囲の外だった

・→ 置き場の数を数え間違えて、隣を踏んでいる

見当 2: 前に一度壊れた仕組み

・関数を呼ぶ前に値を退避し、戻ったら書き戻す

・この作業の途中で、一度ここが壊れていた

・呼び出しの前後で、値が変わって見えた

・→ 呼ばれた側が、書き戻し損ねている

どちらも、証拠つき

そして 2 回とも外れた

見当１を、検査に変える

・コンパイラは関数ごとに「変数を何個ぶん置けるか」を先に決めて確保する

・確保した数と、実際に使った数を、コンパイラ自身に比べさせる

・合わなければ、コンパイルを止める

機械が言った

reserved 2, used 17

関数 gc_collect で、確保 2 個・使用 17 個。疑いは当たっていた

直した

溢れは、どのプログラムでもゼロになった

それでも puts 1 + 1 は通らない

実在の欠陥だった。でも、原因ではなかった

推測のままなら「直し方が悪かったのかも」と迷い続けたはず

検査にしてあったから、当たりと原因を切り離せた

見当 2 も、検査に変える

・CPU エミュレータに、関数呼び出しの約束を見張らせる

・呼ぶ前の値を覚えておき、戻ったときに照合する

一度も鳴らない

検査は、外れも確定させる

外れた分岐が閉じる。だから次に進める

観察が、食い違い始めた

・値を出力するコードを足すほど、結果が互いに矛盾した

・ある監視は「書き換えなし」、別の監視は「書き換えあり」

・推理の結論:「壊れた値は、メモリのどこかにある」

推理をやめて、出来上がった機械語を読む

逆アセンブル = 機械語を、人が読める命令の列に戻すこと

壊れていた値（Ruby が変数を探すための表）を、読みにいく最初の命令は——

mv a0, s10

メモリからではなく、CPU 自身の置き場（レジスタ s10）から取っている

「メモリのどこか」は、丸ごと誤りだった

s10 に書く命令は、その関数に 4 つだけ

うち 1 つが、例外から戻るときの書き戻しだった

例外の仕組み

・失敗するかもしれない処理の前に、今の値を退避する

・失敗したら、退避した値を書き戻す

・退避には置き場所が要るので、メモリを一区画確保する

よくある作り

書いていた先は、区画の外

意図	実際	
確保した区画	15 個ぶん（0〜14）	15 個ぶん（0〜14）
値の退避先	5〜14	20〜29

区画の外は、まだ誰にも渡していない空き地

空き地は、後から払い出される

1. 区画を確保する
2. 値を、区画の外（空き地）に退避する
3. 処理の途中でメモリが要求され、その空き地が払い出される
4. 退避した値の上に、別のデータが書かれる
5. 失敗して書き戻すと、その別のデータが戻ってくる

壊れていたのは 1 行

前: 20 + i × 4　先頭から 20 バイト目

後: (20 + i) × wsz　先頭から 20 個目

32 / 64 ビット両対応のための、機械的な置き換え。1 個 = 8 バイトなので、区画のはるか外

答え合わせ

2. 自作コンパイラが出した機械語

最初の見当「直前に手を入れた場所」は、方向としては合っていた。同じ作業の、別の 1 行だった

では、なぜ緑だったのか

・「例外から戻ると値が戻る」を確かめるテストは、ずっとあった

・退避する値は 8 個。入れ子も、失敗も成功も試していた

・でも、失敗する処理の中でメモリを確保していなかった

確保 0 回なら通る。1 回で壊れる

間違った場所から書き戻すのと

まだ誰も書いていない場所から書き戻すのは

出力が同じ

2 つの検査も、見えない場所にいた

・置き場の数の検査: 関数の区画しか見ない。退避先は別の区画

・呼び出しの約束の検査: 「呼んで、戻る」の対しか見ない。例外の巻き戻しは「戻る」ではない

・どちらも黙っていた。どちらも正直だった

32 ビットも、同じ変更からずっと壊れていた

・症状が出なかっただけ

・64 ビットに移った時点で、32 ビットで回すのをやめていた

・壊れたと気づく手段を、自分で捨てていた

第１章で持ち帰るもの

1. 疑いは、検査にする —— 外れも確定する
2. 黙っている検査を、信じすぎない —— 見えない場所がある
3. 観察が食い違ったら、成果物を読む —— 成果物は構造を述べている

ここから先は、同じ塔で起きた別の事件

第２章　症状は、原因と別の層に出る

事件簿: 文字の d に、横棒が出る

・自作のスライド道具で、30 ピクセルの d の上に、無い線が描かれた

・原因は自作のフォント処理。曲線の向きが変わる点で、線を 1 回多く数えていた

・本物のブラウザとの比較テストは 297 / 297 で通っていた。30 ピクセルの例が無かった

正解を持たない検査

・文字を左右反転して描いても、塗る場所は鏡写しで同じはず

・答えを誰かに用意してもらわなくても、自分自身と比べられる

・古いコードで 87 点、直したコードで 0 点

・最初に見つけたのは、知らなかった組み合わせ（g の 12 ピクセル）

事件簿: 3D の描画が、2 フレーム目で落ちる

・メモリの規則をコンパイラで変えた。3 版続けて出した

・テスト 2,715 本、自作アプリ 29 本の型検査、全部緑

・自作の 3D 描画だけが、2 フレーム目で落ちた

・前のフレームの値を、もう使い回された場所から読んでいた

規則ごと、取り下げた

・「822 MB → 10 MB」と測った改善も撤回した

・そのとき 8 行のテストを足した

・初めて走らせた日に、別の出力先（Wasm）の、生きているバグを見つけた

700 のはずが -424242

事件簿: x86-64 でだけ、9 回目から NaN

・同じ式を 9 回呼ぶと、8 回目までは正しく、それ以降は全部 NaN

・最適化なしでは正しい。arm64 では正しい。C の出力先でも正しい

・x86-64 でだけ、LLVM の出力先でだけ壊れる

・見つけたのは 3D 描画の CI。初めてこの出力を x86-64 で動かしたのがそれだった

原因は、LLVM の最適化の 1 つ

・値を 3 つ組で返す呼び出しに、「末尾呼び出し」の印が付いた

・3 つ組は、arm64 ではレジスタで、x86-64 ではメモリで返る

・8 回目までは正しく、9 回目から崩れる。崖は計算ではなく状態を指す

・最適化パスを 1 つずつ外して、1 つに絞った

事件簿: CI が、生まれてからずっと赤

・コンパイラが出す C の main は、トップレベルの let 1 つごとに括弧が 1 段深くなっていた

・clang の括弧の深さの上限は 256。3D 描画の main は 533 段

・Ubuntu の clang は上限を守る。手元の Apple の clang は守らない

・最初の検査で落ちるので、その後ろの検査は 1 本も走っていなかった

事件簿: 足し算の順番は、C コンパイラのもの

```
p "one" ++ p "two" ++ p "three"
```

・解釈実行、LLVM、clang では one / two / three

・同じ C を gcc でビルドすると three / two / one

・C は、式の中の評価順を決めていない

事件簿: ループにしていたのは、clang だった

・自作言語にはループ構文が無い。繰り返しは、自分自身の呼び出しで書く

・引数 1 つの自己呼び出しを、コンパイラはループにしていなかった

・clang の最適化が、たまたま平らにしてくれていた

・CPU エミュレータにコードを足したら、clang の判断が変わり、全部「スタック溢れ」

同じ日に、もう１つ

・RISC-V の出力先で、辞書の巡回が、入れた順の逆だった

・それが見えたのは、Ruby 処理系の Hash#inspect だけだった

{"MRB_B"=>"2", "MRB_A"=>"1"}

事件簿: CRC は、64 ビットでは偶然正しかった

・自作の圧縮器の検査値（CRC32）は、全部のビットが 1 の状態から始める

・自作言語の右シフトは、符号を保つ

・全部 1 は、32 ビットでは負の数、64 ビットでは正の数

・Ruby 処理系を 32 ビットの CPU で動かして、初めて壊れた。README の「gzip とバイト単位で一致」は、どの検査も確かめていなかった

事件簿: 1 枚の画像が、2 つの違うファイルになる

・自作の PNG 読み込みで、同じ画像から、出力先ごとに違うファイルができた

・コンパイルした出力先では、文字列が C の文字列で、0 のバイトで終わる

・だから 0 のバイトが全部消えた。解釈実行では残った

・短い方のファイルも、画像としては開けてしまう

事件簿: Ruby の定数表が、壊れていた

・自作の CPU の上で Ruby を初めて動かした日

・落ちたのは、文字列の連結の中

・原因は、メモリをまとめて巻き戻す仕組みが、外の辞書の中身まで回収していたこと

・同じプログラムが、解釈実行、C、LLVM、Wasm では全部正しかった

事件簿: CPU 0% で、止まる

・Ruby の公式テストを流すと、途中で止まる。CPU は 0%

・調べると、スレッドが、自分自身が持っているロックを待っていた

・原因は自作言語の実行時ライブラリ。辞書を使い回すとき、区画を本当より大きく 4 KB と主張して、隣を踏んでいた

・壊れた例外メッセージで巨大な確保が失敗し、ロックを持ったまま止まっていた

決め手は、アドレスではなかった

・ASan と、使い回すときにメモリを毒で埋めるビルドを作った

・決めたのは、壊れた場所ではなく「その領域を誰が確保したか」の行だった

・使用後解放に見えて、実ははみ出しだった

事件簿: 同じ処理が 4 つ、それぞれずれていた

- 出力先ごとに、ほぼ同じ変換処理のコピーを持っていた
- 4 つが少しずつ違う方向にずれていた
- Wasm では == が間違い、LLVM では存在しない関数を呼んでいた
- 1 つにまとめた日に、両方消えた

事件簿: popen 1 回で、メモリが 33 MB → 537 MB

・メソッドの中で IO.popen を 1 回呼ぶと、スタックに古い値が 1 語残った

・その語が、たまたま Hash の置き場の番地を指していた

・「指されている区画は詰め直さない」規則で、詰め直しが永久に止まった

・Hash の置き場が 33 → 134 → 537 MB と伸びた

事件簿: 整数の幅が、3 つある

・実行時の整数は 64 ビット

・コンパイラは OCaml で書いていて、OCaml の整数は 63 ビット。2^63 - 1 をそのまま書けない

・clang -O2 は、溢れた後に確かめる検査を消す（溢れは C では未定義）

第３章　２つの誤りが重なると、正しい出力になる

事件簿: 印字は、ずっと正しかった

・コンパイラの印字は、出力先の番号を入れ忘れていた

・CPU エミュレータは、その番号を見ずに全部画面に出していた

・出力は正しい。テストは緑

・エミュレータを本物に近づけた瞬間、印字が壊れた（2026 年 10 月）

コードにはコメントまであった

「エミュレータは番号を無視する。QEMU は無視しない」

そのコメントの所は直してあった。同じ形の印字と失敗の経路、ほかの 5 箇所は直っていなかった

事件簿: 本家 ruby との一致が 2,589 件

・比較には、Ruby のテスト枠組みを真似た自作の部品を使っていた

・その部品は、テストの一部を何もせずに通していた

・両側で同じ部品が動くので、両側とも同じように何もしない。数えると「一致」

何もしていなかったもの

・it_behaves_like: 1,781 行が、何も走らせていなかった

・例外の検査は、クラスだけを見て、メッセージ 1,068 箇所を捨てていた

・「例外が出ないこと」の検査は、存在しなかった（155 箇所、両側でエラー）

・本物どおりに直したら、一致は 2,589 → 2,221。本物の穴も 10 個近く出てきた

直したあとの比較でも、1 回騙された

・最初の比較で、環境変数を 1 つ渡し忘れた

・テストの中の exit! が、比較の部品ごと殺していた

・「集計なし」のほとんどが、計器自身の失敗だった

事件簿: ソケットのテストが、パスの長さを測っていた

・UNIX ソケットのパスは 104 バイトまで。越えるとテストは自分を飛ばす

・作業ディレクトリが 144 バイトの場所から測ると、両側とも飛ばして「8 / 8 一致」

・短いパスから測ると 0 / 8

・標準ライブラリの一致数は 416 → 396。今は 72 バイトを越える場所からの記録を断る

事件簿: 殺されたファイルが、98 / 98 の緑

・Ruby のテスト実行器は、失敗の報告を最後にまとめて出す

・比較の側は「名前が出て、報告に無い」を「通った」と数えていた

・メモリの番人に殺されたファイルは、走った分が全部「一致」になった

直した版の方が、悪く見えた

- 暴走を直したら、test_exception が 98 → 49 に下がった
- 退行を探す前に、古い版を作り直して走らせた
- 古い版は 98 本目で 13 GB まで膨らみ、報告を 1 行も出さずに死んでいた
- 新しい値の方が正しかった

事件簿: 道具が無ければ、成功

・QEMU との比較は「QEMU が入っていなければ飛ばす」と書いてあった

・CI の QEMU のインストールは「失敗しても赤くしない」

・どちらも単独なら妥当。重なると、検査が消えても CI は緑

・数えたら 65 本が同じ形。46 本が CI に居た

それを塞ぐ検査も、最初は自分を通した

・道具が揃っているかを先に確かめる検査を書いた

・最初の版は、名前が直接書いてある道具しか読まなかった

・きっかけの $QEMU を見落とし、「必須 0 個」と言って自分自身を通した

事件簿: どの CI にも居ない検査

・RISC-V の機械語と C の出力を比べる検査は、どの CI にも名前が無かった

・初めて全部走らせたら、赤だった

・1 を 60 ビット左にずらすと、C では 1152921504606846976、32 ビットの CPU では 0

・別の問いのために足したテストが、この検査を赤にしたまま放置されていた

事件簿: 全部走らせたら、3 本赤

・CI が名前を挙げる検査 124 本を、初めて全部走らせた

・3 本赤。2 本は公開済みの main で赤。1 本は 20 版以上前から赤

・毎回「関係しそうな検査」だけ走らせていた

・その直後、CI 自体も 4 版続けて赤だった（CI の機械に dune の道が通っていなかった）

事件簿: 毒見が、黙って力を失う

・毒見 = わざと壊して、検査が赤くなるか確かめること

・メモリの配置を変えた版で、ある毒見が赤くならなくなった。検査は正しいまま

・別の毒見は、63 回中 2 回、緑になった。どちらも機械が重いとき

・解放した場所が、たまたま再利用されなかった

毒見は、壊して測る

・「解放 → 再利用 → 上書き」に頼る毒見は、配置と割り当て器の気分に左右される

・割り当て器自身に、解放時に中身を壊させた

・39 本全部が赤。正しい版は緑のまま

事件簿: ランダムでは、壊れない

・自作の圧縮器が、32×32 のランダムな画素の PNG で落ちた

・ランダムなバイト列 1〜4096 バイト、種 3 つでは、全部通った

・壊れたのは、PNG が実際に作るデータの形のときだけ

・その 3,104 バイトを、そのままテストに入れた

事件簿: 答え合わせの相手が、何も確かめていなかった

・本家 ruby で通るテストを、自作 Ruby が落とした

・足りない警告を足す前に、本家で 1 本だけ走らせて、検査の数を見た

・本家は 0 本の検査で「通って」いた。eval の中の return がテストのメソッドごと抜けていた

・本当の差は、自作 Ruby の eval の return だった

事件簿: 落ちるはずのない検査

・Wasm の出力に「この命令が含まれるはず」と書いた検査が 41 本あった

・確かめると、11 本は言っていることが偽だった

・i32.mul を探していたが、利用者のコードにあるのは i64.mul

・確かめ方が空虚なので、言っていることが間違っていても緑になっていた

事件簿: データベースの検査が、黙って飛ぶ

- PostgreSQL の検査は、手元で静かに SKIP になっていた
- postgres は icu4c 74 を求め、Homebrew は 78 に上げていた
- postgres が起動できず、検査 3 本が、走らずに成功していた

2 つの誤りが重なると、正しい出力になる

・書き戻し先がずれている × まだ誰も上書きしていない

・番号を入れ忘れた × 番号を見ない

・部品が何もしない × 両側で同じ部品

・パスが長い × 両側で同じように飛ばす

比較が見ているのは、出力だけ

第４章　答え合わせの相手にも、層がある

事件簿: 本家 ruby は、掛け算と足し算を 1 命令にしていた

・1.0.step(2.0, 0.1) の値が、本家と 1 ulp ずれた

・本家の ruby は clang でビルドされていて、i × unit + beg を fma 命令 1 つにまとめていた

・自作言語の a × b + c は、まとめない

・合わせるには、明示的に fma を呼ぶしかなかった

事件簿: macOS の数学関数は、正しく丸めていない

・同じ 502 点で、macOS は 39 点、Linux（glibc 2.39）は 4 点だけ、正しい値からずれた

・自作 CPU の上の答えは、70 桁の bc で確かめると正しかった

・Mac の ruby の方が 1 ulp ずれていた

・基準を「手元の数学ライブラリのビット」から「正しく丸めた値」に移した

事件簿: 本家 ruby の版と機械も、答えを変える

・ruby 3.4 は Hash#inspect の => の前後に空白を入れた

・3.2.11 は、3.2.2 が出していた警告を出さなくなった

・ソケットの定数は、macOS と Linux で違う

・だから当時の CI は、答え合わせを macOS の ruby 3.2.2 に固定していた（今は 4.0.6）

答え合わせの相手は、下の層ではない

本家 ruby も、clang でビルドされ、手元の数学ライブラリの上で動く、1 つの C プログラム

「下の層は正しい」は、答え合わせの相手にも仮定

第５章　遅さも、別の層にいる

事件簿: 圧縮しない圧縮器

・自作の圧縮器のテストは「元に戻るか」「検査値が合うか」だけだった

・何も圧縮しなくても、全部通る

・コンテナの層（8.9 MB）を圧縮させたら 369 秒。gzip は 0.1 秒

・毎回すべての距離を総当たりしていた。直して 1.3 秒（250 倍）

同じ圧縮器の、ほかの 2 つ

・圧縮できないと判断した塊を、まるごと無圧縮で書いていた。10,000 バイトが 10,023 バイトに（gzip は 1,366）

・入力を全部いっぺんに 2 回分解していた。9 MB の入力で、使用メモリの最大が 489 MB

・1 GiB の機械で動かす予定のデーモンだった

事件簿: このスライドが、1 枚 25 秒

・10 月、この発表のスライドを作ったら、33 枚で 14 分かかった

・動いているプログラムを覗くと、時間は全部、あの圧縮器の総当たりの中

・直したはずの圧縮器は、PNG の部品が古い版に固定したままだった。2 つ下のライブラリ

・固定を上げて、33 枚で 3 秒。画素は 1 つも変わらない

事件簿: Hash 1 つに、索引 1 つ

- Ruby で Hash を 100 万個作って捨てる。キー 9 個で 2,288 MB
- 重さは Ruby 処理系ではなく、自作言語の側にあった
- Hash ごとの索引が、二度と返らない共有の領域に置かれていた
- 小さな Hash は索引を持たない（CRuby と同じ考え）。2,288 MB → 201 MB

事件簿: 安全の検査が、1 時間終わらない

・止まった処理が指しているメモリを回収しないよう、検査を足した

・検査 1 回は安い。でも Ruby 処理系は、メソッドから戻るたびに訊いていた

・止まっている処理は最大 1,000 個。csv のベンチマークが 1 時間で終わらない

・最初の直し（結果を覚える）は、時間を並べ替えに移しただけだった

問いを狭める

・区画には作られた時点の、止まった処理には止まった時点の印を付けた

・区画ができる前に止まった処理は、その区画を指しようがない

・だから、区画より後に止まった処理だけを調べる

・元の速さの 4〜16% 増しまで戻った

検査の値段は、呼ばれる頻度で決まる

事件簿: 計画に書いた原因は、2 つとも外れた

・見当 1:「浮動小数点の二分探索が、違う値を試している」

・検査を素の Ruby に書き写して比べると、3,196 通り全部、同じ答え

・見当 2:「この変数がメモリを掴んでいる」

・外しても、メモリも物の数も変わらない

本当の原因

- 本当は、捕まえたフレームの 4 KB の区画が、置き場に返っていなかった
- 80,000.times { lambda } が 827 MB → 149 MB
- ついでに ruby -e 1 の起動が 244 MB → 46 MB（本家は 19 MB）

事件簿: 1 行で、169 MiB → 3.5 MiB

- 二分木を作っては捨てるベンチマーク
- 「この中で作ったものは、ここを出たら全部捨ててよい」と 1 行書いた
- 169 MiB → 5.8 MiB。さらに型の印を指し先に詰めて 3.5 MiB
- 比較: Rust 5.5、C 3.3、Go 11.8 MiB

でも、LLVM の出力先では回収していなかった

・同じプログラムが、C の出力先では 2.5 MB、LLVM の出力先では 316 MB

・出力は同じなので、出力先どうしの比較テストには見えない

・直して 5.8 MB

事件簿: 自作の CPU で、命令を数える

・CPU エミュレータに、命令ごとの実行回数を数えさせた

・コーパス 165 本 × 10 回で、16 億 3,400 万命令

・辞書の操作が 61%。機械語で書き直し、その他いくつかを直して 8 億 5,800 万命令（−47%）

・次の計画は、数えたら全体の 1% だと分かって、やめた

事件簿: gsub 10 万回で 1 GB

・gsub を 10 万回呼ぶと 4.18 秒、1,048 MB

・1 マッチごとに 1 つずつ漏れていた

・デバッガが遅すぎるので、生成した C に目印を埋めて、誰が作ったかを引いた

・1.76 秒、181 MB

事件簿: 測って、作ったものを捨てた

・プロファイルは「メソッドの表引きが 12%」と言った

・表引きの結果を覚える仕組みを作った

・7 回ずつ走らせて、最速はどちらも 3.71 秒

・戻した

事件簿: 時間切れではなく、メモリの番人だった

・あるテストが 41 → 0 本になった。600 秒の時間切れだと思った

・時間で何度も比べた

・殺していたのは、6 GB を越えたら殺す番人。元の版も 5.9 GB で、すれすれだった

・番人の記録を、最初に見るべきだった

その後

・同じテストのメモリの最大は 6.1〜6.5 GB → 2.4 GB

・回収のたびに、今まで作った文字列と Hash の番号を全部調べ直していた

・CPU 時間 490 秒 → 73 秒

事件簿: 1 GB のファイルを開く

・自作のテキストエディタで 1 GB のファイルを開くと 822 ms → 8.6 ms

・改行のたびに索引を全部作り直していた。Enter と取り消しが 730 ms → 3.2 ms

・計測の道具は、最初「↓キーに 823 ms」と言った

・起動の途中から測っていた。数字が悪いほど、まず計器を疑う

事件簿: 測ったのは、古いファイルだった

・Wasm の出力の大きさを測った。632 KB、「十分小さい」

・測ったのは、もう作られていない古いディレクトリだった

・本物は 1,064,718 バイト。使わない関数を刈り取って 616,025 バイト（−42%）

・違うものを測っても、測ったようにしか見えない

事件簿: 1,000 個の関数が、1 つの塊

- Ruby 処理系を C に変換するのに 113 秒
- 処理系は、互いに呼び合う約 1,000 個の関数の 1 つの塊
- コンパイラの 4 つの処理が、関数の数の 2 乗で遅くなっていた
- 113 秒 → 14 秒

第６章　開発を進めたいだけなのに

Ruby 処理系の開発は、こう回る

・処理系を直す

・本家 ruby の仕様テスト（ruby/spec）を全部流して、本家と比べる

・一致が増えたか、減ったかを見る

その 1 周が、遅くなっていった

連鎖 1: 仕様テストの 1 周が 90 分

・静かな機械で 90 分。混んだ機械で 4 時間

・1 日に何周もできない。開発が止まる

・まず、処理系を速くしなければ、と思う

推測しないで、1 本を測った

時間

実時間	2.71 秒
処理系が使った CPU	0.15 秒
OS の中で使った時間	1.89 秒

遅いのは処理系ではなかった

テストの道具が、ファイルをコピーしていた

・テストのファイル 1 本ごとに、変わらない 4,333 個のファイルを丸ごとコピーしていた

・1 周で約 1,000 万回のコピー

・macOS のファイル監視が、その後何時間も 100% を越え、6 GB を使い続けた

・ほかの作業まで最大 8 倍遅くなり、それを処理系の止まりだと思って 1 時間失った

1 周に 1 回だけ作る

90 分 → 30 分

記録は 1 バイトも変わらない

次: 10 コアのうち、0.58 しか使っていない

・6 並列にした

・30 分 → 9 分

・ところが、同じバイナリで、記録が 3 通り出た

制限が、全部壁時計だった

- 時間切れの判定が、実時間で測られていた
- 並列で機械が混むと、同じテストが切られたり、切られなかったりする
- CPU 時間の制限（25 秒）に変え、実時間はその 24 倍まで待つ
- メモリの番人も、掃き自身が起動するようにした

まだ揺れる: シグナルが受け継がれていた

・シェルは、裏で走らせる仕事に「Ctrl-C を無視する」設定を渡す

・並列の子プロセスは全部、それを受け継いでいた

・Ctrl-C を自分に送って確かめるテストの振る舞いが変わっていた

・直して 1,829.7 秒 → 661.6 秒。記録の行は 1 つも動かない

並べ方の工夫は、2 案とも遅かった

- 大きいものから先に流す: 719.9 秒
- 10 並列にする: 736.7 秒
- 6 並列で、使っているのは 10 コアのうち 2.09。その半分が OS の中
- 律速は、共有のファイルシステムだった

次: ビルドの検査 29 秒のうち 22 秒

- 「使われていない定義がないか」の検査
- 定義 1 つごとに awk を 1 回起動していた
- 1,950 万回の行の走査
- 1 回の awk にまとめて 0.64 秒。検査全体は 29 秒 → 4.95 秒

次: 残りはコンパイラの中

・Ruby 処理系を C に変換するのに 113 秒

・処理系は、互いに呼び合う約 1,000 個の関数の 1 つの塊

・型を調べるとき、名前を連想リストで探していた。9 万回の検索、平均の深さ 1,184

・4 つの処理が 2 乗。113 秒 → 14 秒。出力 5,045 個はバイト単位で同じ

1 日で追った順番

1. テストの道具（ファイルのコピー）
2. OS（ファイル監視）
3. 並列化
4. 時間の測り方
5. シグナル
6. ビルドの検査（awk）
7. コンパイラ（型検査の 2 乗）

処理系を速くしたかったのに、処理系はずっと最後だった

連鎖 2: テストのファイルが、測れない

・本家の Ruby のテスト（test/unit）を流すと、16 本が 6 GB を越えて番人に殺された

・殺されたファイルは、一致したかどうかすら分からない

・速さ以前に、測れない

原因は 1 つではなかった

- 文字列を ++ で継ぎ足す 2 乗: 4 万要素の Array#join が 10 GB → 238 MB
- メソッドやブロックの中では、回収が 1 度も走らない作りだった
- test/unit は、ファイル全体を at_exit のブロックの中で走らせる
- だから 1 本の間、Ruby のオブジェクトは 1 つも回収されない

番人は、ビルドまで殺していた

・6 GB の番人は、引数に「mere-ruby」を含むプロセスを全部見ていた

・mere-ruby をビルドする clang も、出力先の名前に mere-ruby を含む

・clang は 6 GB を越える。掃きの最中にビルドすると「暴走」として殺された

回収を、別のコルーチンで走らせる

・回収の間、処理系のスタックは「止まったスタック」になる

・Fiber のために作った道具が、そのまま使える

・そうしたら、Fiber にもあった穴が 2 つ出てきた。穴のある版では、一致が 3,471 → 1,609 に落ちた

・自作言語の側で穴を直して、3,527

次々に出てくる、2 乗

・最後の回収の 1 段が 2 乗だった

・配列に 1 つ書くたびに、配列全体をコピーしていた: 15 GB → 162 MB

・文字列の組み込みが 2 乗: 100 MB の bytes に 12〜16 GB

1 つ直すと、次のファイルが最後まで走り、次の 2 乗が見える

連鎖 3: 自作 CPU の上の Ruby が、時間切れ

・Ruby のサンプル 165 番が、自作 CPU の上で 89 秒。6 並列だと 120 秒を越える

・3 本が時間切れ

・CPU エミュレータに、命令ごとの実行回数を数えさせた

数えた順に、直す

・辞書の検索が命令の半分 → 機械語の手続きに。全体で −47%

・次は起動だけで 3 億 9,000 万命令。1 本 4 秒、サンプルを 1 周 19 分

・文字列比較の 89% がリテラルとの比較 → その場に展開: 3 億 4,900 万

・遠くへの分岐を、近いものは短く: 3 億 6,490 万 → 3 億 1,170 万（−14.6%）

最後は、CPU エミュレータ自身

・エミュレータは、ゲストの 1 バイトにホストの 8 バイトを使っていた

・256 MB のゲストに、2.2 GB

・1 バイトは 1 バイトに: 386 MB

・だからゲストに 1 GB を渡せるようになり、メモリ不足だったサンプルが一致した

ボトルネックは、毎回、別の層にいた

・テストの道具、OS、シグナル、awk、コンパイラ

・Ruby 処理系、自作言語の実行時ライブラリ、CPU エミュレータ

・全部自分のコードだと、次のボトルネックも自分のコード

測らずに直すと、毎回違う層を直していたはず

第７章　１つ直すと、次ができるようになる

連鎖 1: 自作 CPU の上に、自作 OS（8 月 11 日、1 日）

・自作言語にはループ構文が無い。繰り返しは、自分自身の呼び出しで書く

・末尾呼び出しをループにしないと、止まらないプログラム（OS）が書けない

・まず末尾呼び出し、メモリの回収、メモリが尽きたら言うヒープ

作っていたら、壊れていたものが出る

・調べる途中で、自作 CPU の上でのセルフホストのデモが、もう壊れていたと分かった

・メモリの配置を RAM の大きさから決めるようにした

・自作コンパイラが、自作 CPU の上で、また 1 バイトも違わない出力を出した

1 つずつ積む

1. 生のメモリを、偽造できない値として渡す。画面への出力（UART）
2. UART を書いたら、ビット演算が無いと分かった → ビット演算、CPU の制御レジスタ
3. キー入力 = 「シェルに要る最後の部品」
4. 割り込みとタイマー → 「割り込みから戻る先」がスケジューラになる
5. 横取りするスケジューラ（新しい部品 5 つ）
6. シェル（普通のコードで書けた）
7. ユーザプロセス（言語に足したのは 1 つだけ）

そして、その日の最後に

自作コンパイラが、自作 OS の上の普通のユーザプロセスとして動いた

タイマーに 3,727 回割り込まれながら

本来のコンパイラと 1 バイトも違わない出力を出した。新しく要ったものは無かった

連鎖 2: Fiber が、本家のテストを動かした

・Ruby の Fiber を、まず OS のスレッドで作った。Fiber の仕様テストは 2 / 13 → 13 / 13

・でも、Fiber が止まっている間は、何も回収できない

・自作言語に、同じスレッドの中で切り替わるコルーチンを足した

・止まったスタックが指している区画は回収しない、という規則も

Fiber のための道具が、回収を救った

・1 時間終わらない安全の検査を、問いを狭めて直した（第 5 章）

・Fiber と Thread が、どちらもコルーチンの上で動くようになった

・その道具で、回収を別のコルーチンで走らせた（第 6 章）

・本家のテストのファイルが、殺されずに最後まで走るようになった

一致が、段で増える

いつ	何をしたか	一致
10 / 2	回収を別のコルーチンで（途中の版は 1,609 に落ちた）	3,471 → 3,527
10 / 3	読み込みで止まっていた 5 本が走る	3,899
10 / 4	手続きの本体を 1 回だけ持つ（test_array 0 → 345）	4,232
10 / 6	外からのシグナルが trap に届く（test_process 0 → 83）	
10 / 14	5,898	
10 / 18	6,304	

連鎖 3: 自作 CPU の上で、Ruby が 1 本ずつ動き出す

・最初は、プログラムを -e で渡すしかなかった。サンプル 162 本中 124 本
・ファイルを読めるようにした: 135
・辞書の使い回しの不具合を直した: 137
・浮動小数点のライブラリ: 142 / 162

ホストの機能を、1 つずつ渡していく

自作 CPU に足したもの	一致
ファイルの削除	226 → 234 / 279
コルーチン（Ruby の Fiber がそのまま動く）	235 → 240
ディレクトリと環境変数	242 → 248
作業ディレクトリ	249
ソケット	254
子プロセス（Linux の形で）	257 → 263 / 283
1 GB の RAM（と本家 ruby 4.0.6 への切り替え）	284 / 290
296 / 302	

残りの内訳（285 / 291 の時点）

・3 本: Linux として正しい違い。自作 CPU の上の Ruby は Linux の答えを、本家は macOS の答えを出す

・1 本: 本家の方が 1 ulp ずれている（macOS の数学関数、第 4 章）

・2 本: 時間切れ。RAM と時間を足せば一致する

・自作言語の側の残りは、無い

直すと、次ができる。できると、次の穴が見える

・OS を作ると、セルフホストの壊れが見えた

・Fiber を作ると、回収の道具ができ、本家のテストが走り、次の 2 乗が見えた

・CPU に機能を渡すと、Ruby のサンプルが動き、次に足りないものが名指しされた

第８章　パッチワーク: Ruby の中の Perl

Ruby も、パッチワーク

Its creator, Yukihiro "Matz" Matsumoto, blended parts of his favorite languages (Perl, Smalltalk, Eiffel, Ada, and Lisp) to form a new language

ruby-lang.org「About Ruby」

Ruby を自作すると

Perl から来た布地を、1 枚ずつ自分で縫い直すことになる

事件簿: $~ は、マッチしたフレームのもの

・最初、$~（最後のマッチ）は、1 つのグローバル変数だった

・メソッドの中のマッチが呼び出し元に見え、呼び出し元のマッチがメソッドに見えた

・呼び出しのたびに退避して、戻す

・ただし、ブロックはフレームではない。それが Ruby の規則

10 個を退避すると、42% 遅い

・$~ と $1〜$9 の 10 個を呼び出しのたびに退避したら、呼び出しの多いコードで 42% 遅くなった

・$1〜$9 は置き場を持つのをやめ、読まれたときに $~ から取り出すことにした

・退避は 1 回で済み、fib(24) は元の速さに戻った

事件簿: 2 行目から 3 行目

```
ruby -ne 'print if 2..3'
```

・awk から来た書き方。2 行目から 3 行目までを出力する（flip-flop）

・最初は 2 と 3 がただの「真」で、全部の行が出力された

・整数の端は $.（今の行番号）と、正規表現の端は $_（今の行）と比べる

・本家の -e の仕様テストが、全部一致するようになった

小さな布地たち

- if /re/ は、何も書かなくても $_ と照合する
- $_ はスレッドごと。$-a、$-l、$-p は読み取り専用
- $; を使うと、非推奨の警告が出る
- BEGIN は最初に、END は終了時に走る
- __END__ は、ヒアドキュメントの中ではプログラムを終わらせない
- $< は ARGF、$> は $stdout、$* は ARGV

継ぎ目を見つけたのは、本家のテスト

・どの布地も、本家の ruby と ruby/spec が「違う」と言ってくれた

・自分で書いた塔の継ぎ目を、他人が書いた判定が見つけた

第９章　この発表自身

9 月、この発表の証拠を測り直した

・棚卸しに書いた「Ruby 処理系は 10 万行」は、どの数え方でも再現しなかった。実測 62,665 行

・境界のテストに挙げた 3 つのうち 1 つは、どの CI にも居らず、測ったら赤だった

・どちらも、自分に甘い方向に外れていた

そして、その訂正も

・9 月、「Ruby 層の 162 本一致は誤り」と訂正した

・今月、CI のログを読み直したら、誤りだったのは訂正の方だった

・162 本という数が、別の比較に 2 つあった

記憶ではなく、ログが判定した

第 10 章　持ち帰ってほしいこと

自分以外が判定する形には、系譜がある

・1988 年、Perl 1.0 の t/TEST。Larry Wall が ok 1 / not ok 1 を数え始めた。TAP の始まり

・Raku は、仕様そのものをテスト集（roast）として定義した

・2006 年、Rubinius のために、Ruby の実行できる仕様が書かれ始めた。今の ruby/spec

全部自分のコードの塔で、最後に頼ったのは、他人が書いた判定だった

自分が正しいと思っていることを
自分以外が判定できる形に置き換える

3 つのやること

やること	なぜ効くか
疑いは検査にする	外れも確定する
黙っている検査を信じすぎない	見えない場所がある
観察が食い違ったら、成果物を読む	成果物は構造を述べている

当てるためではなく、外すために効く

いちばん効いたのは、検査が自分の仮説を否定したとき

自分が書いていない層でも

・「下の層は正しい」は、いつでも仮定

・フレームワークでも、データベースでも、クラウドでも、答え合わせの相手でも

・仮定が外れたときにやることは、変わらない

継ぎ当ては、引きはがれる

・古い布に当てた新しい布は、縮んで引きはがれる。Larry Wall が引いた言葉

・20 バイト目が 20 個目になった 1 行も、直した圧縮器が届かなかった固定も、そうだった

・だから縫い目は、自分以外に確かめてもらう

ありがとうございました

- Mere: https://merelang.org/
- CPU エミュレータ: https://github.com/284km/memu
- Ruby 処理系: https://github.com/284km/mere-ruby
- この資料: https://284km.github.io/decks/